

Brief History of Scaling Uber

**HS Editor**

14 Mar 2024 — 15 min read

Share

This blog post was written by [Josh Clemm](#), Senior Director of Engineering at Uber Eats. This is a repost from his LinkedIn article, approved by the author.

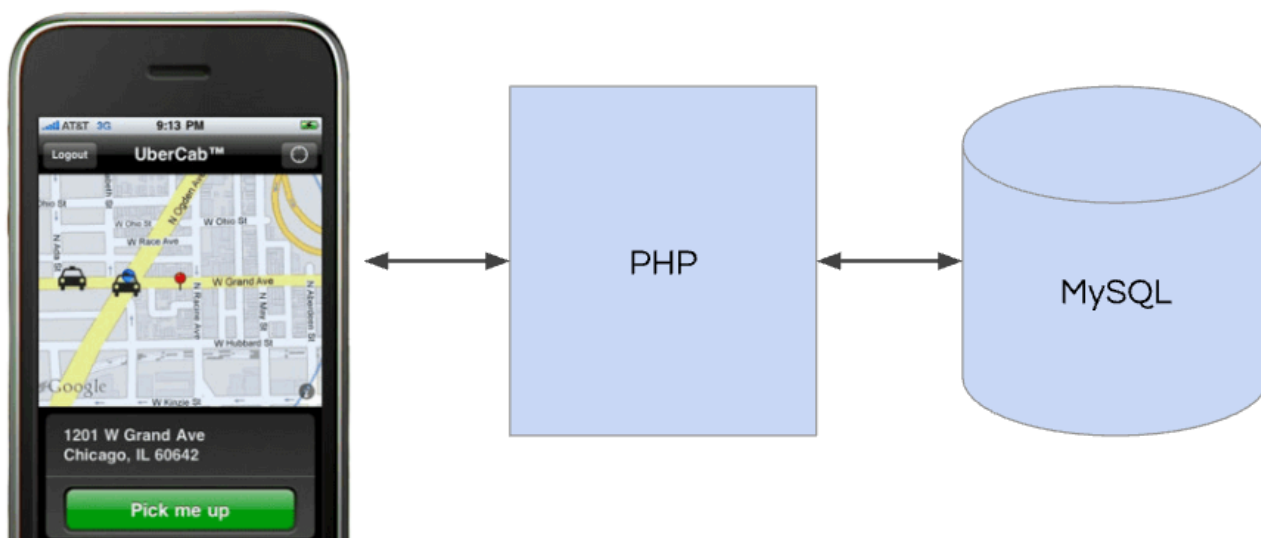
On a cold evening in Paris in 2008, Travis Kalanick and Garrett Camp couldn't get a cab. That's when the idea for [Uber was born](#). How great would it be if you could "push a button and get a ride?"

Fast forward to today where Uber is the [largest mobility platform](#) in the world. It operates in over 70 countries and 10,500 cities. Uber Eats is the [largest food delivery platform](#) ex-China in the world. It operates across 45 countries. We connect millions of driver-partners and merchants with over 130 million customers. We offer dozens of services to [go anywhere or get anything](#). We handle billions of database transactions and millions of concurrent users across dozens of apps and thousands of backend services.

So, how did we get there?

Back in 2009, Uber hired contractors to build the first version of Uber and launched it to friends in San Francisco soon after. The team used the classic [LAMP stack](#) to build the first version of Uber and the code was written in Spanish.

Subscribe



Original LAMP stack proved out the use case, but couldn't scale.

The scaling challenges started as more people wanted to use Uber. There were often major concurrency issues, where we'd dispatch two cars to one person or one driver would be matched to two different riders. (*Learn more about [Uber's earliest architecture from founding engineer Curtis Chambers](#)*).

But the product caught on. It was time to build the tech foundations from scratch.

Setting up for Global Scale

Circa 2011

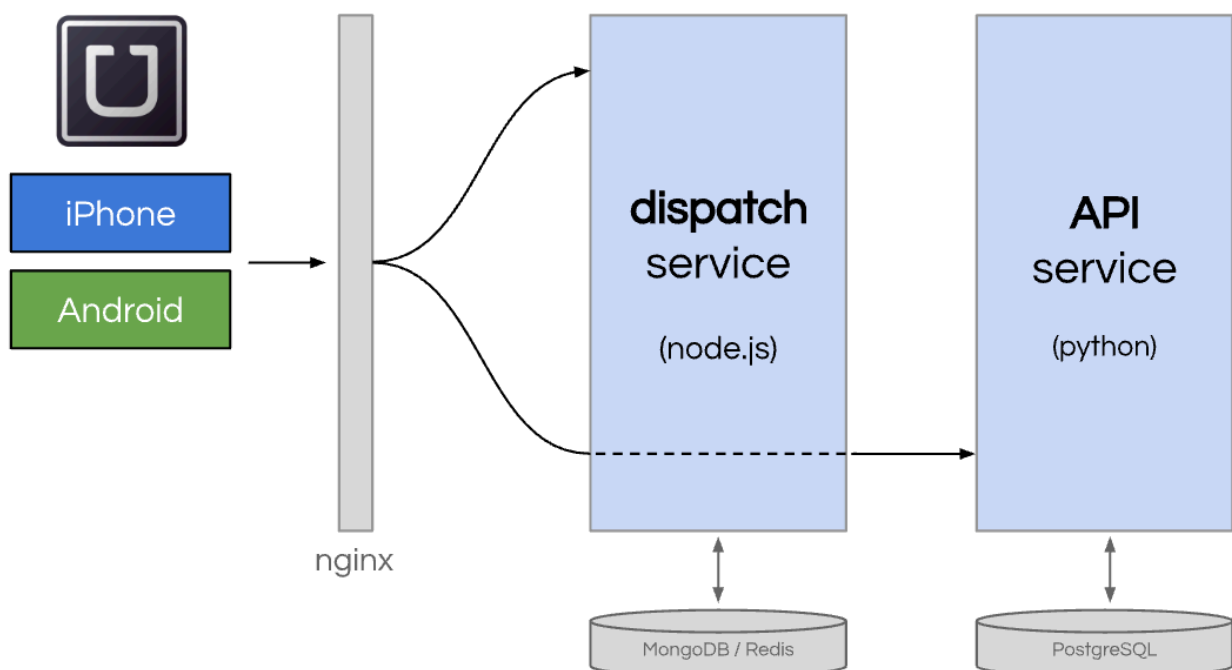
To architect a better and more scalable solution, we needed to address those concurrency problems. Additionally, we needed a system that can process tons of real-time data. Not only are there requests from riders, but Uber needs to track driver real-time locations in order to match riders as efficiently as possible. Finally, the product was still early and would require a lot of testing and iteration. We needed a solution to solve all these scenarios.

Uber adopted [Node.js](#) for their real-time needs. And ended up being [one of the first major adopters of Node.js](#) in production. Node.js was ideal for a few reasons. First, Node.js handles requests asynchronously (using a non-blocking, single-threaded event loop) and so can process significant amounts of data quickly.

Second, Node.js runs on the [V8 JavaScript engine](#), so not only is it performant, but it's excellent for quick iteration.

Uber then created a second service built in Python to handle business logic functions like authentication, promotions, and fare calculation.

The resulting architecture was two services. One built in Node.js ("dispatch") connected to MongoDB (later Redis) and the other built in Python ("API") connected to PostgreSQL.



Uber's two monolith architecture allowed the engineering org to begin to scale

And to improve the resiliency of Uber's core dispatching flow, a layer between dispatch and API known as "ON" or Object Node was built to withstand any disruptions within the API service. (*Learn more about Uber's earliest efforts to maintain service reliability in this video [Designing for Failure: Scaling Uber by Breaking Everything](#)*).

This architecture started to resemble a service oriented architecture. Service oriented architectures can be very powerful. As you carve out services to handle more dedicated functionality, it has a side benefit of allowing easier separation of engineers into dedicated teams. Which then allows for more rapid team scaling.

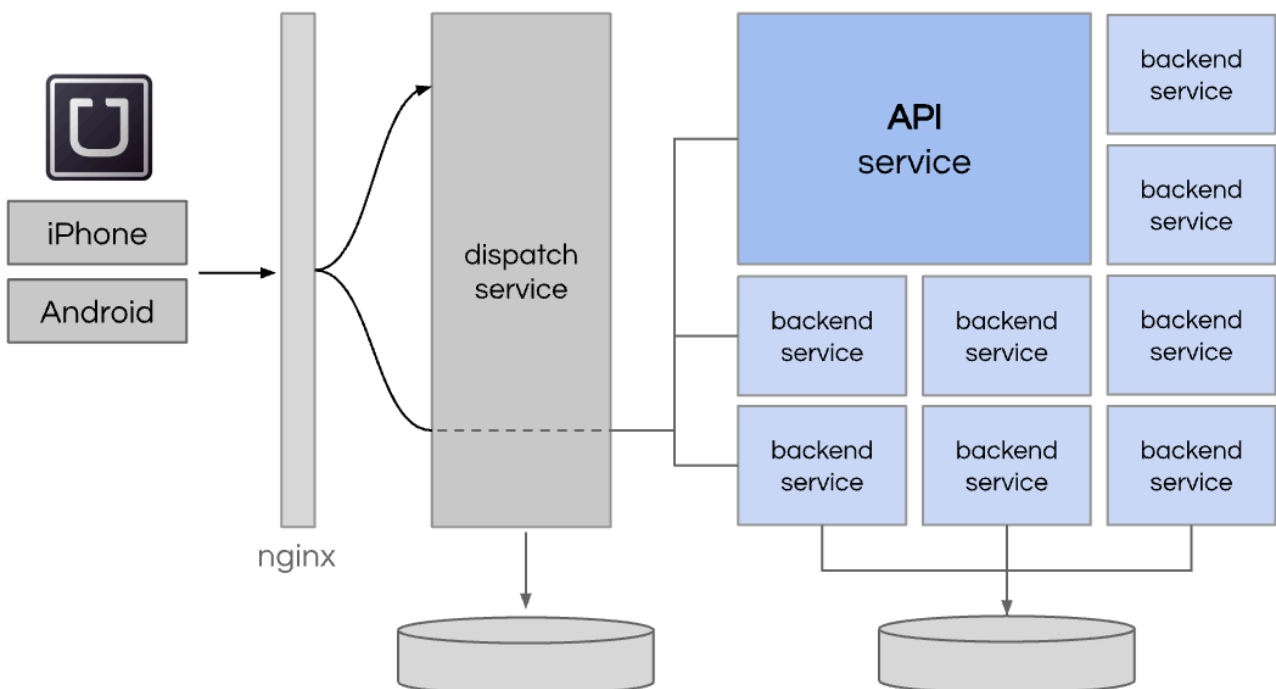
But as the team and number of features grew, the API service was getting bigger and bigger. More and more features were conflicting with one another. Engineering productivity was slowing down. There were huge risks continuously deploying the codebase.

It was time to split out **API** into proper services.

From Monolith to Microservices

Circa 2013

To prepare for our next phase of growth, Uber decided to [adopt a microservice architecture](#). This design pattern enforces the development of small services dedicated to specific, well-encapsulated domain areas (e.g. rider billing, driver payouts, fraud detection, analytics, city management). Each service can be written in its own language or framework, and can have its own database or lack thereof. However, many backend services utilized Python and many started to adopt [Tornado](#) to provide asynchronous response functionality. By 2014, we had roughly 100 services.



Uber's API monolith to microservice migration

While microservices can solve many problems, it also introduces *significant operational complexity*. You must only adopt microservices after [understanding the tradeoffs](#) and potentially build or leverage tools to counteract those tradeoffs. And if you don't consider the operational issues, you will simply create a [distributed monolith](#).

Here's some examples of the issues microservices create and what Uber did to address.

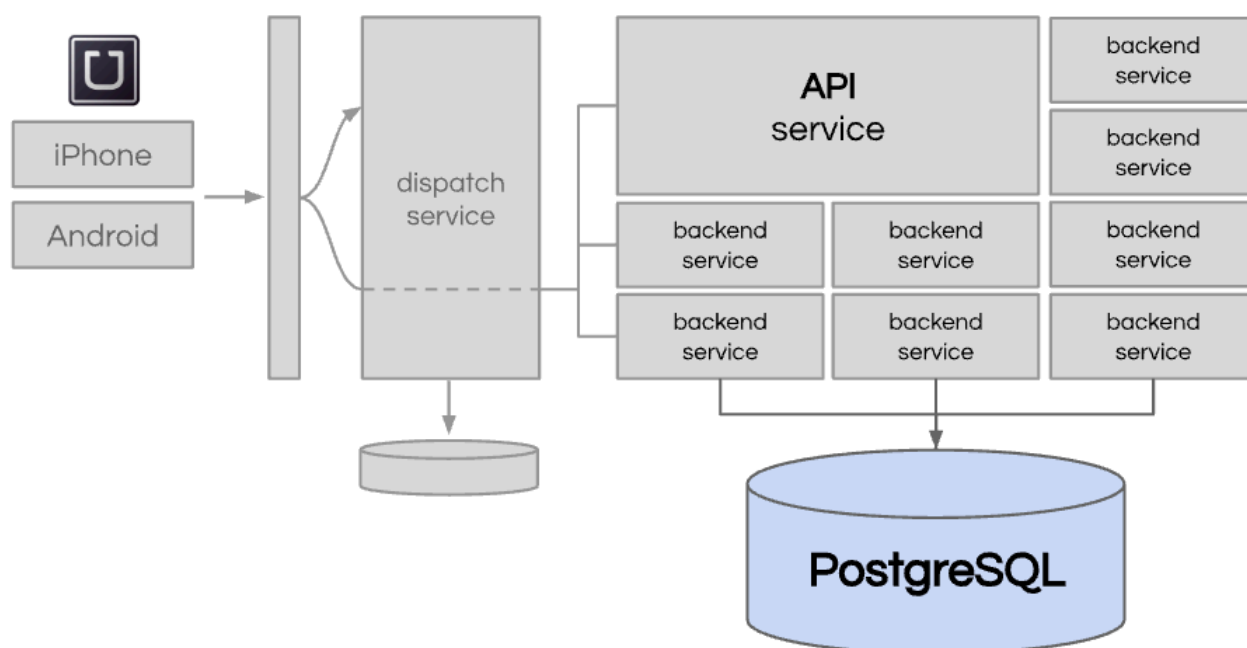
- To ensure all services use **standardized service frameworks**, we developed [Clay](#). This was a Python wrapper on [Flask](#) to build restful backend services. It gave us consistent monitoring, logging, HTTP requests, consistent deployments, etc.
- To **discover and talk to other services** and provide service **resilience** (fault tolerance, rate limiting, circuit breaking), Uber built [TChannel](#) over [Hyperbahn](#). TChannel as our bi-directional RPC protocol was built in-house mainly to gain better performance and forwarding for our Node and Python services, among other benefits.
- To ensure **well-defined RPC interfaces** and stronger contracts across services, Uber used [Apache Thrift](#).
- To prevent **cross-service capability** issues, we use [Flipr](#) to feature flag code changes, control rollouts, and many other config-based use cases.
- To improve the **observability** of all service metrics, we built [M3](#). M3 allows any engineer easy ways to observe the state of their service both offline or through Grafana dashboards. We also leverage [Nagios](#) for alerting at scale.
- For **distributed tracing** across services, Uber developed [Merckx](#). This pulled data from a stream of instrumentation via Kafka. But as each service started to introduce asynchronous patterns, we needed to evolve our solution. We were inspired by Zipkin and ultimately developed [Jaeger](#), which we still use today.

Over time, we've migrated to newer solutions like [gRPC](#) and Protobuf for [interfaces](#). And many of our services utilize Golang and Java.

Scaling the Trip Database

Circa 2014

While Uber was creating many new backend services, we continued to use one single PostgreSQL database.

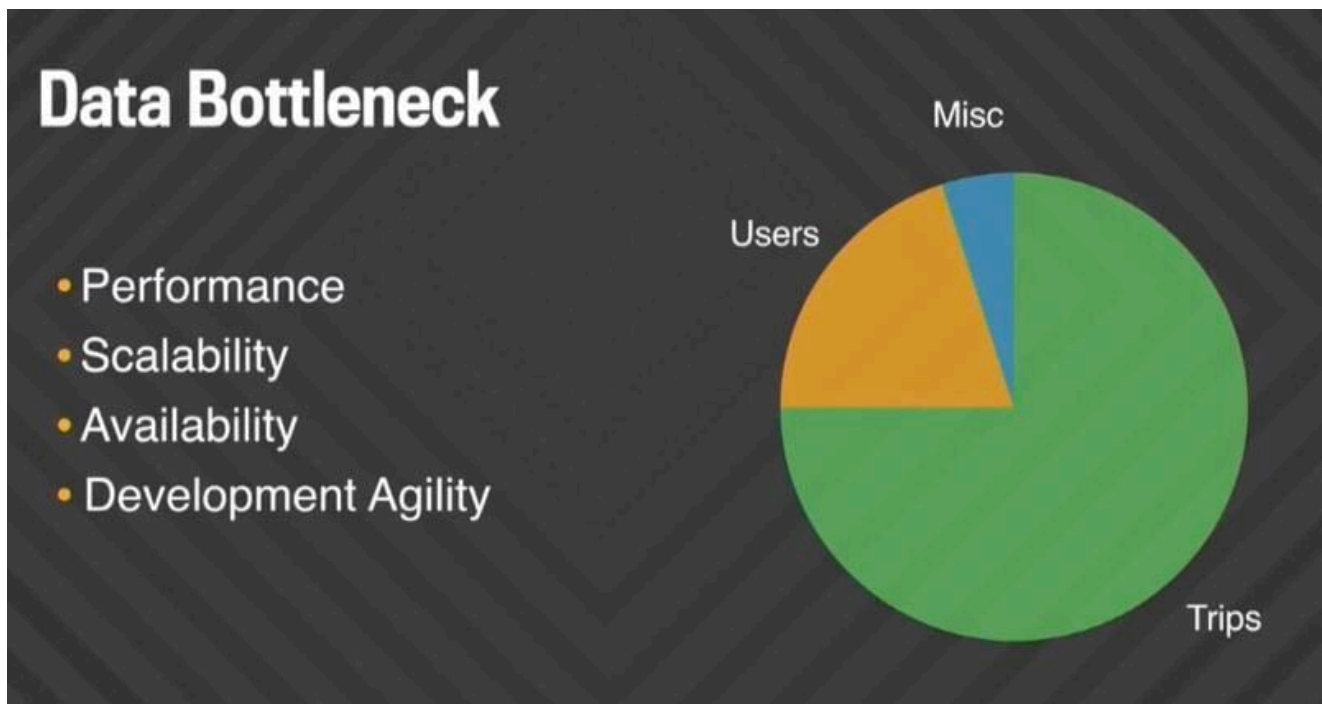


The single PostgreSQL DB became a bottleneck

We were hitting some significant issues. First, the performance, scalability, and availability of this DB was struggling. There was only so much memory and CPUs you could throw at it. Second, it was getting very hard for engineers to be productive. Adding new rows, tables, or indices for new features became problematic.

And the problem was getting existential. By early 2014, Uber was 6 months away from [Halloween night](#) - one of the biggest nights of the year. We needed a more scalable solution and needed it fast.

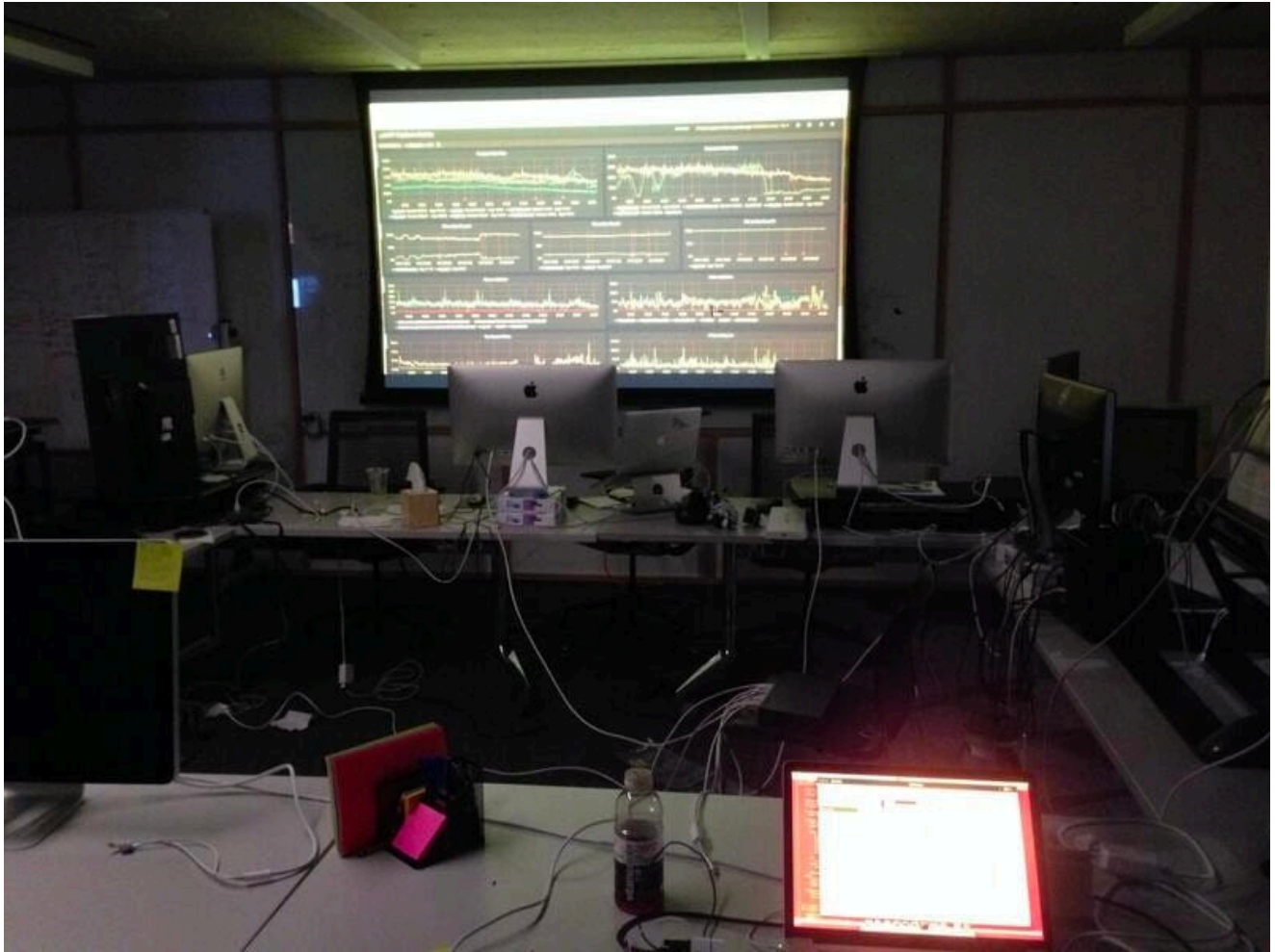
When we looked into the data mix of this DB, the majority of storage was related to our trips, which was also growing the fastest.



The mix of data stored in our single PostgreSQL DB in early 2014

We use trip data in order to improve services like Uber Pool, provide rider and driver support, prevent fraud, and develop and test features like suggested pick-ups. So we embarked on developing [Schemaless](#), our new trip data store. Schemaless is an append-only sparse three dimensional persistent hash map, similar to Google's Bigtable, and [built on top of MySQL](#). This model lent itself naturally to horizontal scaling by partitioning the rows across multiple shards and supported our rapid development culture.

And we successfully [migrated all our services](#) that access trip information in time to avert the Halloween peak traffic disaster. (*Learn more with this [video from lead engineer Rene Schmidt about our creation of and migration to Schemaless](#)*).



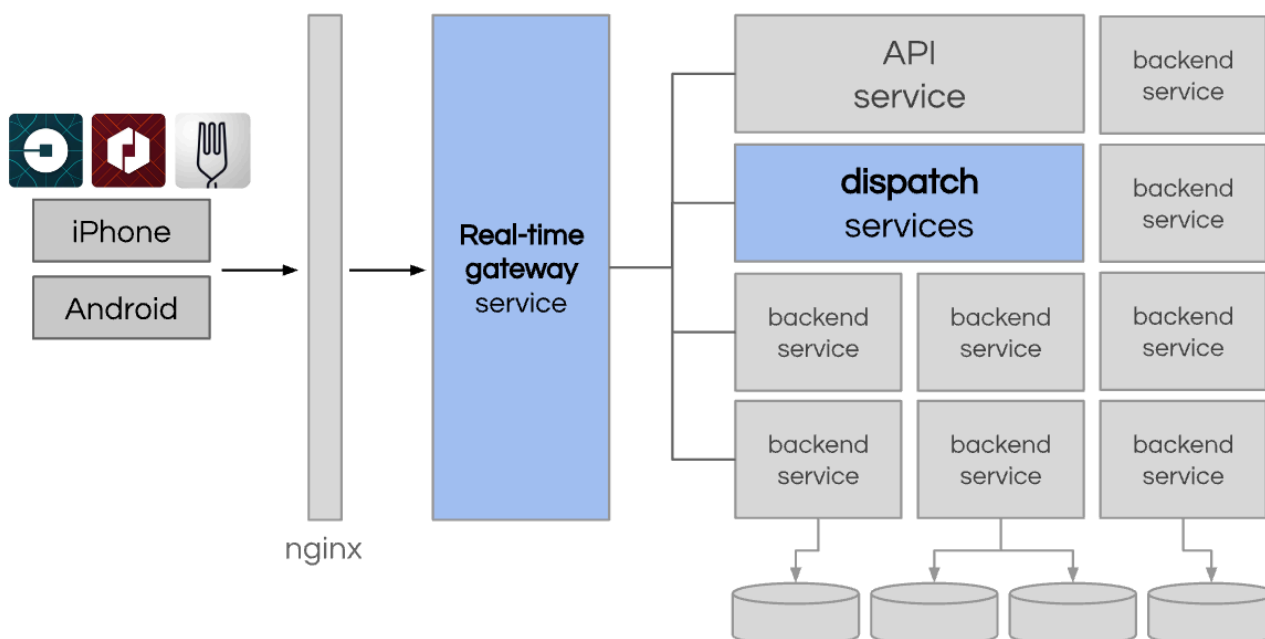
The Schemaless migration operational room - a common look into migrations at scale

While we used Schemaless for our trip data store, we started to use [Cassandra](#) as a replacement for our other data needs, including the database that we use for marketplace matching and dispatching.

Splitting up Dispatch

Circa 2014

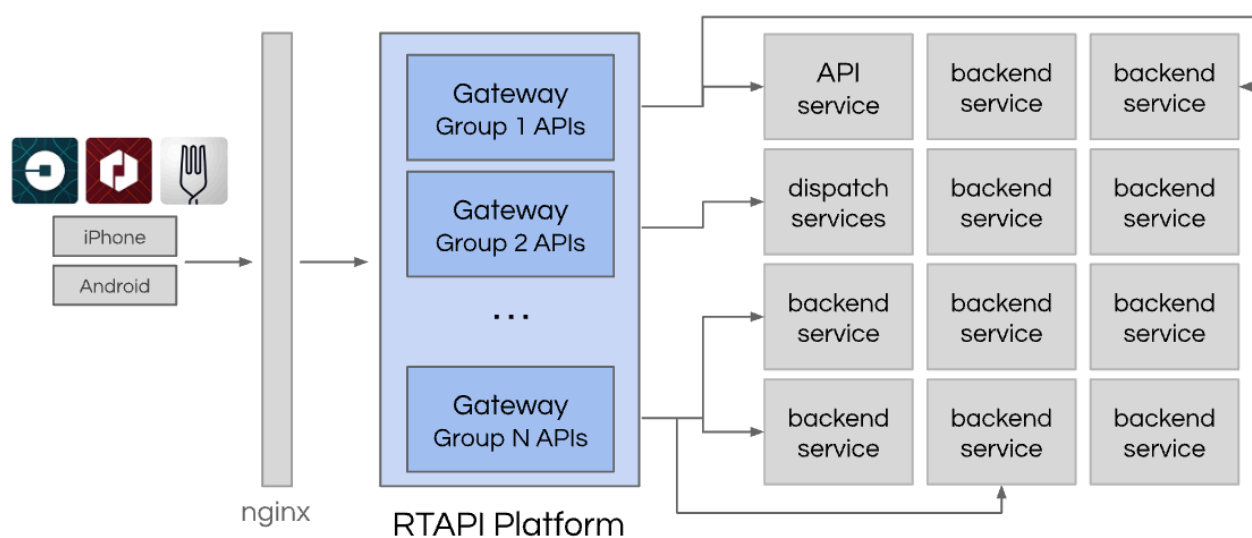
Among Uber's original two monoliths, we discussed the evolution of API into hundreds of microservices. But dispatch similarly was doing too much. Not only did it handle matching logic, it was the proxy that routed all other traffic to other microservices within Uber. So we embarked on an exercise to split up dispatch into two areas of cleaner separation.



Splitting the monolithic dispatch service into a real-time API gateway and an actual dispatch service

Extracting Uber's Mobile Gateway from Dispatch

To better handle all the real-time requests from our mobile apps, we created a [new API gateway layer named RTAPI](#) ("Real-Time API"). And we continued to use Node.js for it. The service was a single repository that was broken up into multiple specialized deployment groups to support our growing businesses.



RTAPI provided a powerful new real-time layer that maintain high development velocity

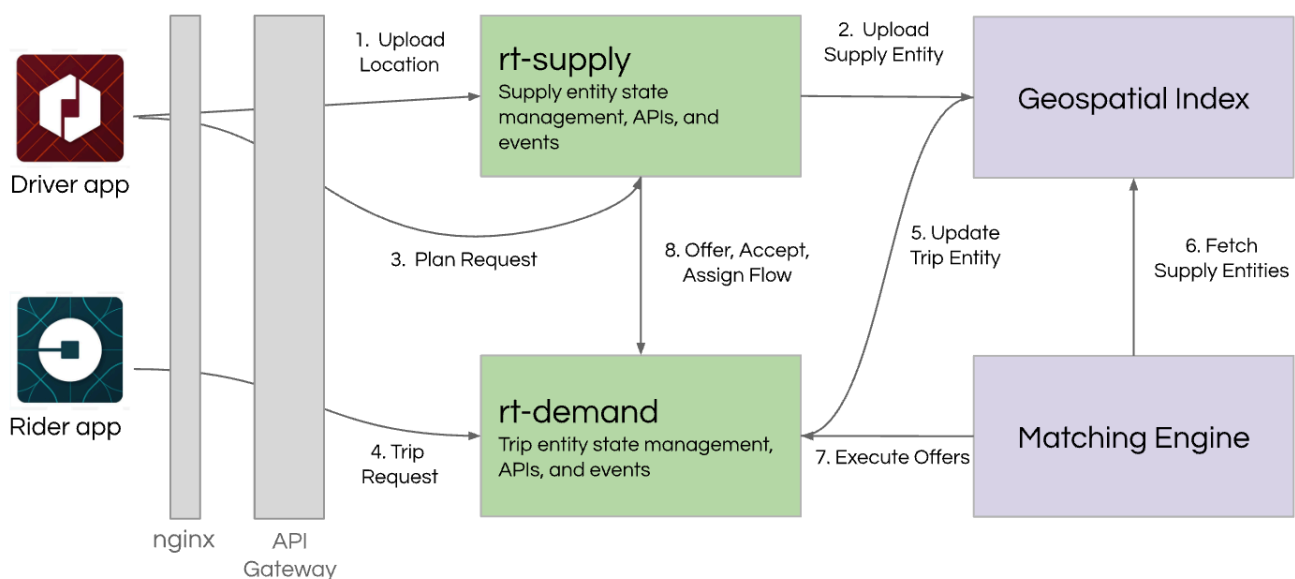
The gateway provided a very flexible development space for writing new code and had access to the hundreds of services within the company. For instance, the first generation of Uber Eats was completely developed within the gateway. As the team's product matured, pieces were moved out of the gateway and into proper services.

Rewriting Dispatch for Uber's Growing Size

The original dispatch service was designed for more simplistic transportation (one driver-partner and one rider). There were deep assumptions that Uber only needed to move people and not food or packages. Its state of available driver-partners was sharded by city. And some cities were seeing massive growth of the product.

So, dispatch was rewritten into a series of services. The new dispatch system needed to understand much more about the types of vehicles and rider needs.

It took on advanced matching optimizations, essentially to solve a version of the [traveling salesman problem](#). Not only did it look at the ETAs of currently available driver-partners, but needed to understand which drivers could be available in the near future. So we had to build a geospatial index to capture this information. We used [Google's S2 library](#) to segment cities into areas of consideration and used the S2 cell ID as the sharding key. *(We've since updated to and [open-sourced H3](#))*



Overview of dispatch stack

Since these services were still running on Node.js and were stateful, we needed a way to scale as the business grew. So we developed [Ringpop](#), a gossip-protocol based approach to share geospatial and supply positioning for efficient matching.

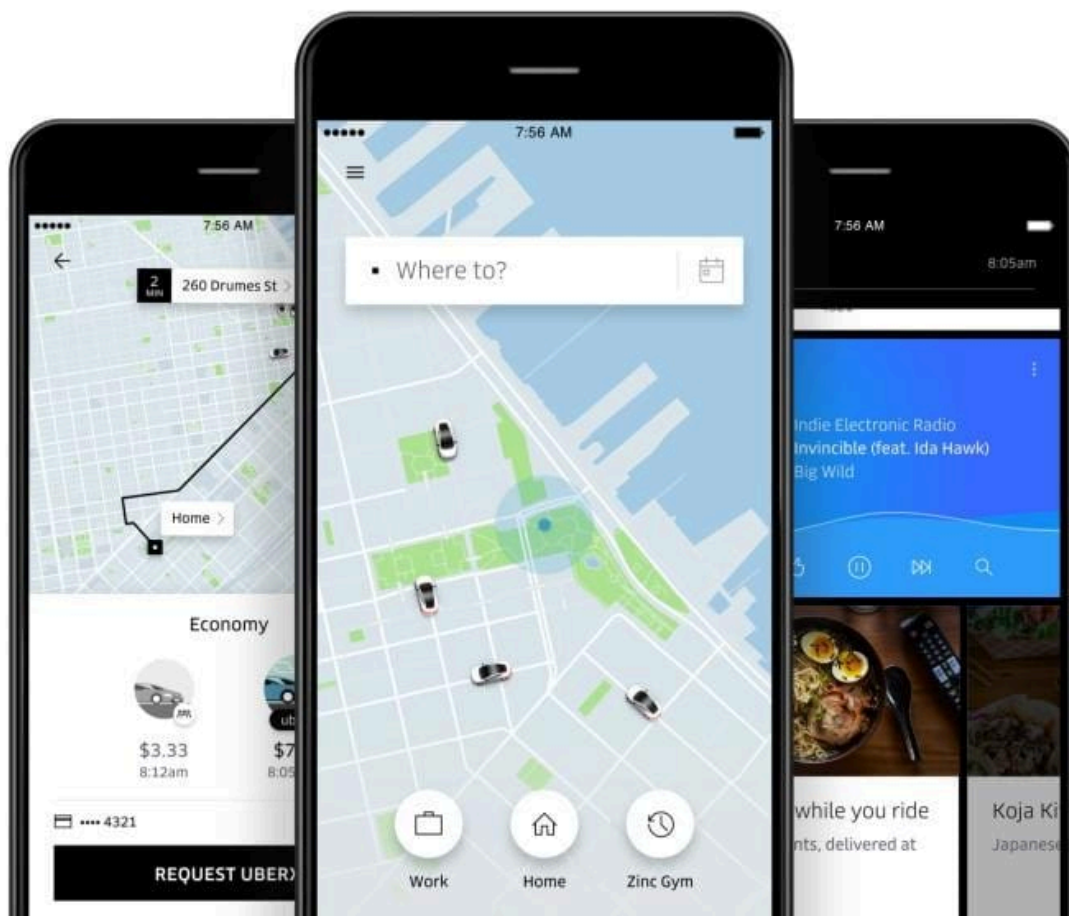
Learn more about [the history of our dispatch stack](#) here or watch this [video on Scaling Uber's Real-time Market Platform](#).

Mobile App Development at Scale

Circa 2016 to present

The flagship Uber product could only have existed due to the new mobile paradigm created by the iPhone and Android OS launches in 2007. These modern smartphones contained key capabilities like location-tracking, seamless mapping, payment experiences, on-device sensors, feature-rich user experiences, and so much more.

So Uber's mobile apps were always a critical part of our scaling story.

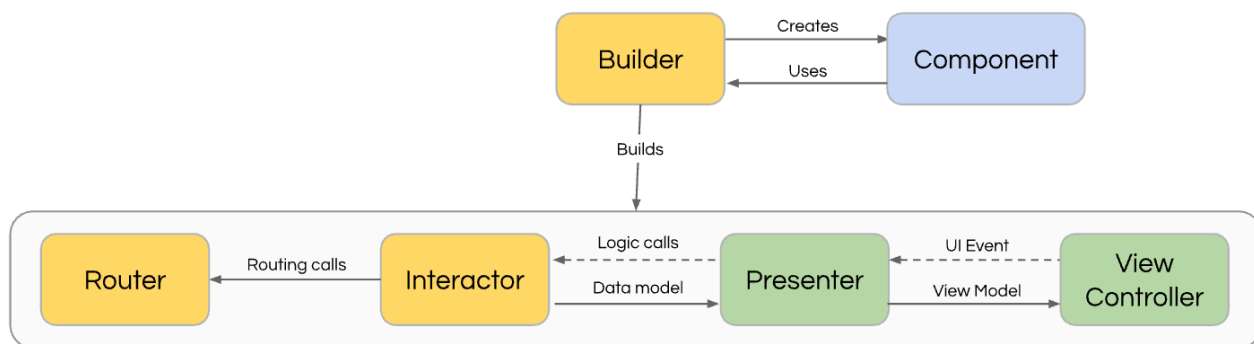


The Uber rider app was critical in defining a scalable mobile architecture

As Uber scaled across the globe, there was a need for an ever-growing list of features. Many of these were specific to certain countries like localized payment types, different car product types, detailed airport information, and even some newer bets in the app like Uber Eats and Uber for Business.

The mobile app's repositories slowly hit similar bottlenecks to a backend monolith. Many features and many engineers, all trying to work across a single releasable code base.

This led Uber to develop the [RIB architecture](#) for mobile, starting with the [rewrite of the main Uber app](#).



RIB stands for Router, Interactor, Builder

Like the benefits of microservices, RIBs have clear separation of responsibilities. And since each RIB serves a single responsibility, it was easy to separate them and their dependencies into core and optional code. By demanding more stringent review for core code, we were more confident in the availability of our core flows. And this allows simple feature flagging to ensure the app continues to run reliably.

And like microservices, RIBs can be owned by different teams and engineers. This allowed our mobile codebases to easily scale to hundreds of engineers.

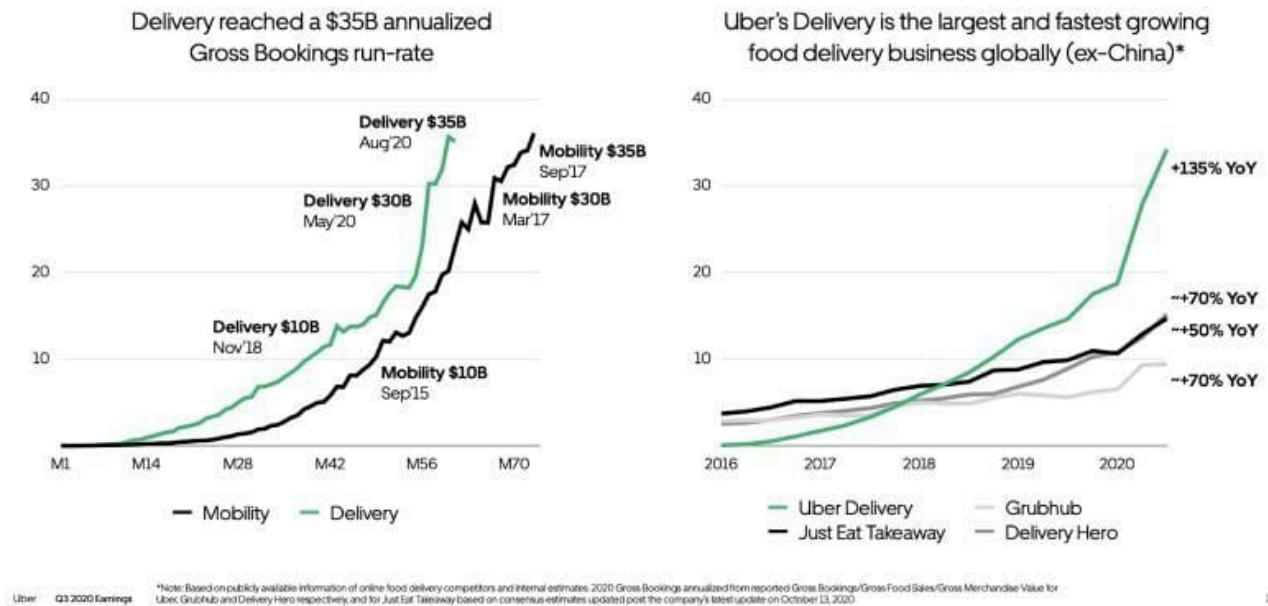
Today, all our apps have adopted RIBs or are migrating towards it. This includes our main [Driver](#) app, the Uber Eats apps, and Uber Freight.

The Rise of Uber Eats

Circa 2017

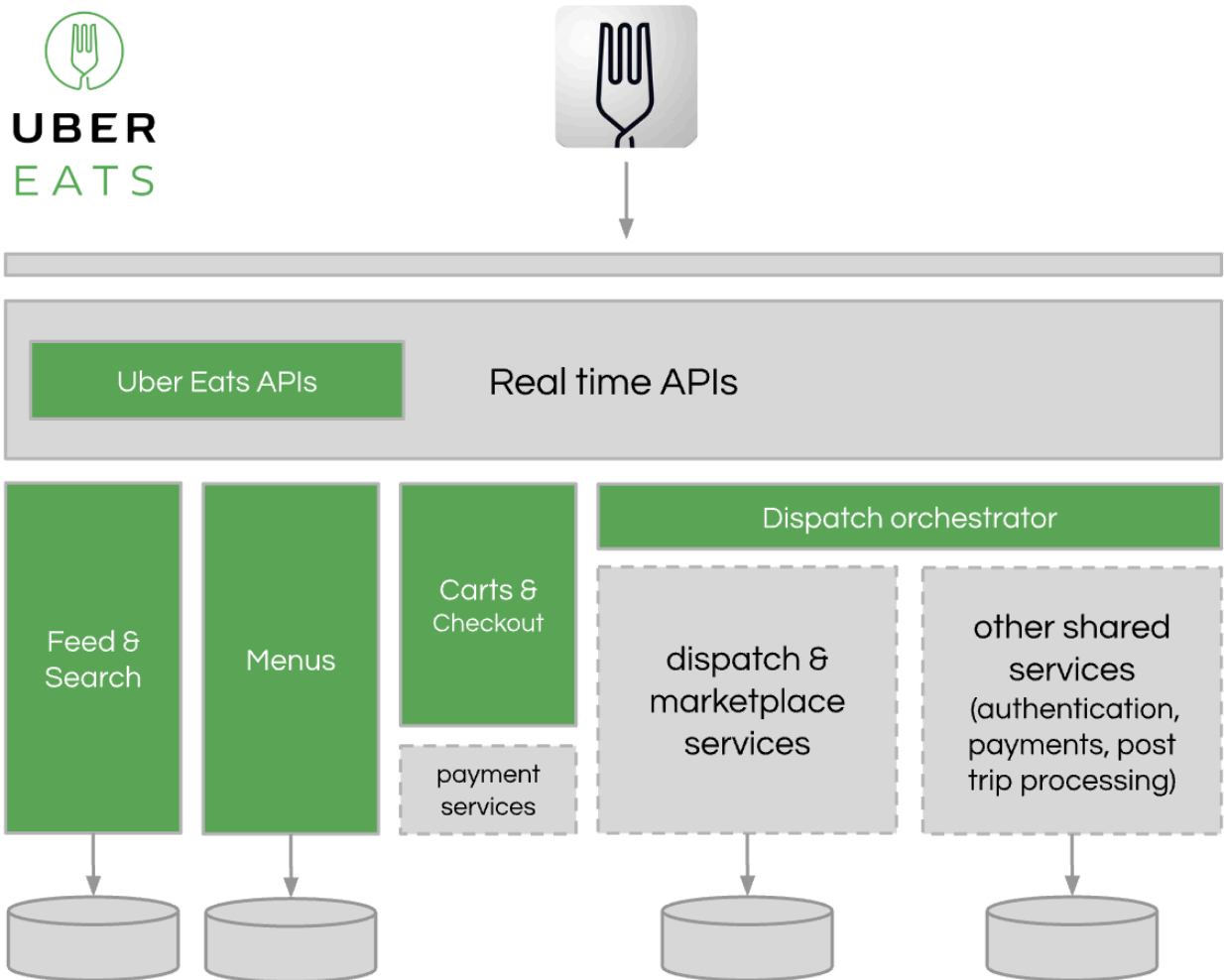
Uber had been experimenting with a number of “Uber for X on-demand” concepts since 2014. And all early signs pointed towards food. So in late 2015 Uber Eats [launched in Toronto](#). And followed a [similarly fast growth trajectory](#) just like UberX.

Delivery outgrowing competition at much larger scale



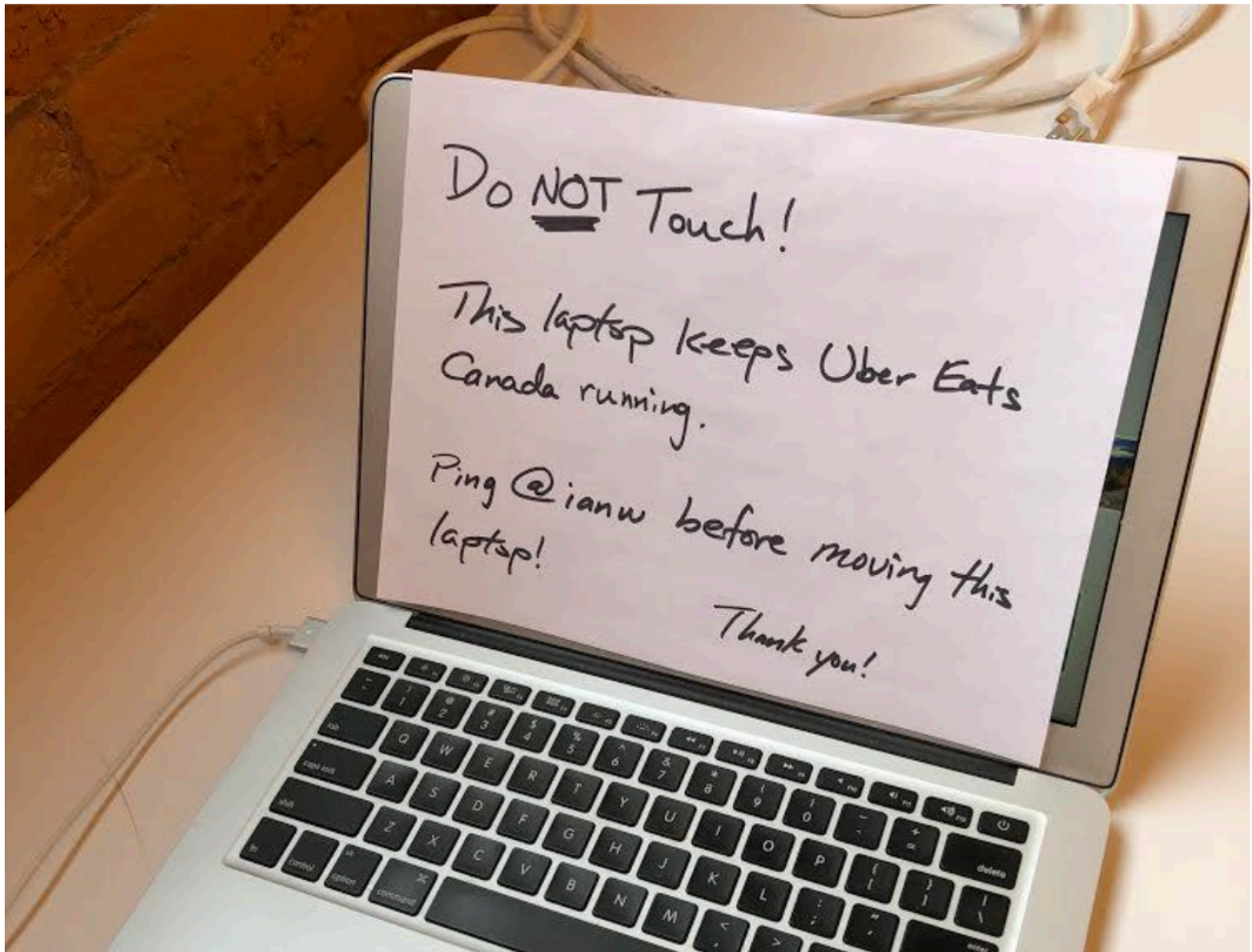
Uber Eats business growth compared with Uber rides (Uber Q3 2020 Earnings)

To enable this rapid growth, Uber Eats leveraged as much of the existing Uber tech stack as possible, while creating new services and APIs that were unique to food delivery (e.g. e-commerce capabilities like carts, menus, search, browsing).



A simplified view into early Uber Eats architecture and how it leveraged systems built for original Uber

The operations team that needed to tune their cities' marketplace often got creative and did things that didn't scale (until the appropriate tech was built).



Uber Eats Canada running scripts to help manage which stores were active and tuning the delivery radius of each based on available driver partners

Early Uber Eats was "simple" in that it supported a three-way marketplace of one consumer, one restaurant, and one driver-partner. Uber Eats today (130+ million users, dozens of countries) supports a variety of ordering modes and capabilities and can support 0-N consumers (eg. guest checkout, group ordering), N merchants (eg. multi-restaurant ordering), and 0-N driver partners (eg. large orders, restaurants which supply their own delivery fleet).

The history of how Uber Eats evolved probably deserves its own scaling story and I may one day get to it.

But for now, to learn more from the earliest days, I highly recommend listening to Uber Eats founder Jason Droege's recount of "[Building Uber Eats](#)".

Bring on the Standards - Project Ark

Circa 2018

No scaling story is complete without understanding the context and culture of the company. As Uber continued to expand city by city, local operations folks were hired to ensure their city launch would go successfully. They had tools to ensure their local marketplace would remain healthy and the flywheel would grow. As a result, Uber had a very distributed and decentralized culture. And that helped contribute to Uber's success in getting to 600 cities by 2018.

That culture of decentralization continued within engineering, where one of our earliest cultural values was "Let Builders Build". This resulted in rapid engineering development that complemented Uber's success growing across the globe.

But after many years, it resulted in the proliferation of microservices (thousands by 2018), thousands of code repositories, multiple product solutions solving very similar problems, and multiple solutions to common engineering problems. For example, there were different messaging queue solutions, varying database options, communication protocols, and even many choices for programming languages..

"You've got five or six systems that do incentives that are 75 percent similar to one another" - Former Uber CTO Thuan Pham

Developer productivity was hurting.

Engineering leadership recognized it was time for more standardization and consistency and formed [Project Ark](#). Project Ark sought to address many aspects of engineering culture that contributes to scaling:

- Engineer productivity,
- Engineer alignment across teams,
- Duplication,

- Unmaintained critical systems, and
- Knowledge access and documentation.

As a result, we elevated Java and Go as official backend languages to [gain type-safety and better performance](#). And deprecated the use of Python and Javascript for backend services. We embarked on reducing code repos from 12,000 down to just our main languages (Java, Go, [iOS](#), [Android](#), and web). We defined more standardized architectural layers where client, presentation, product, and business logic would have clear homes. We introduced abstractions where we could group a number of services together (service "[domains](#)"). And continued to standardize on a series of service libraries to handle [tracing](#), logging, networking protocols, resiliency patterns, and more.

A Modern Gateway for the Modern Uber

Circa 2020

By 2019, Uber had many business lines with numerous new applications (Uber Eats, Freight, ATG, Elevate, and more). Within each line of business, the teams managed their backend systems and their app. We needed the systems to be vertically independent for fast product development.

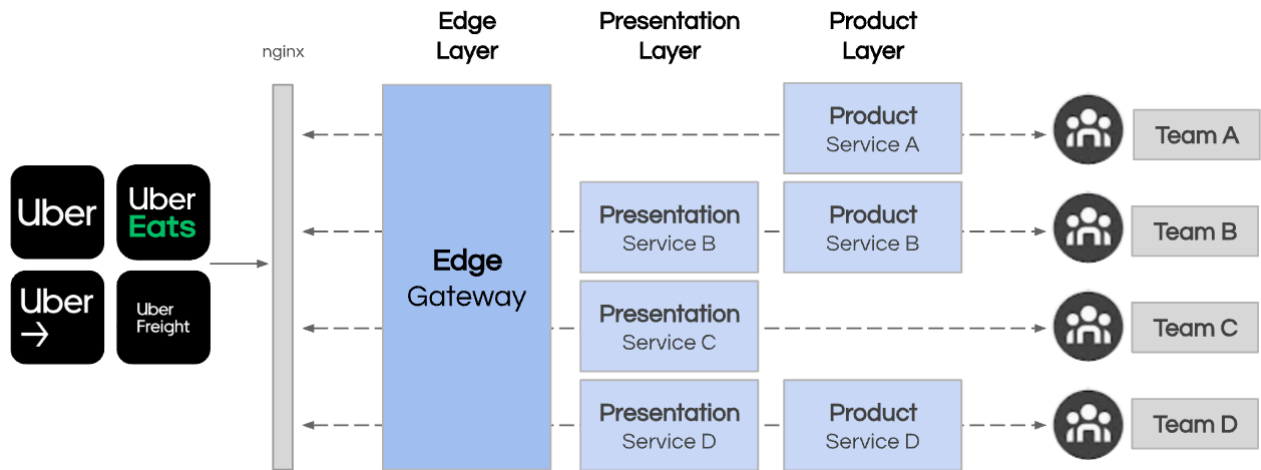
And our [current mobile gateway was showing its age](#). RTAPI had been built years ago and continued to use Node.js and Javascript, a deprecated language. We were also eager to make use of Uber's newly defined architectural layers as the ad hoc code added to RTAPI over the years was getting messy with view generation and business logic.

So we built a new [Edge Gateway](#) to start standardizing on the following layers:

- Edge Layer: [API lifecycle management layer](#). No extraneous logic can be added keeping it clean.
- Presentation Layer: microservices that build view generation and aggregation of data from many downstream services.
- Product Layer: microservices that provide functional and reusable APIs that describe their product. Can be reused by other teams to compose and build

new product experiences.

- Domain Layer: microservices that are the leaf node that provides a single refined functionality for a product team.



This evolution set us up well to continue building new products with velocity, yet with the necessary structure to align our 1000s of engineers.

Next Generation Fulfillment

Circa 2021

Throughout the years, Uber has created a world-class platform for matching riders and driver-partners. So our dispatch and fulfillment tech stack is a critical part of Uber's scaling story.

By 2021, Uber sought to power more and more delivery and mobility use cases. The fulfillment stack was showing its age and couldn't easily support all these new scenarios. For example, we needed to support reservation flows where a driver is confirmed upfront, batching flows with multiple trips offered simultaneously, virtual queue mechanisms at Airports, the three-sided marketplace for Uber Eats, and delivering packages through Uber Direct.

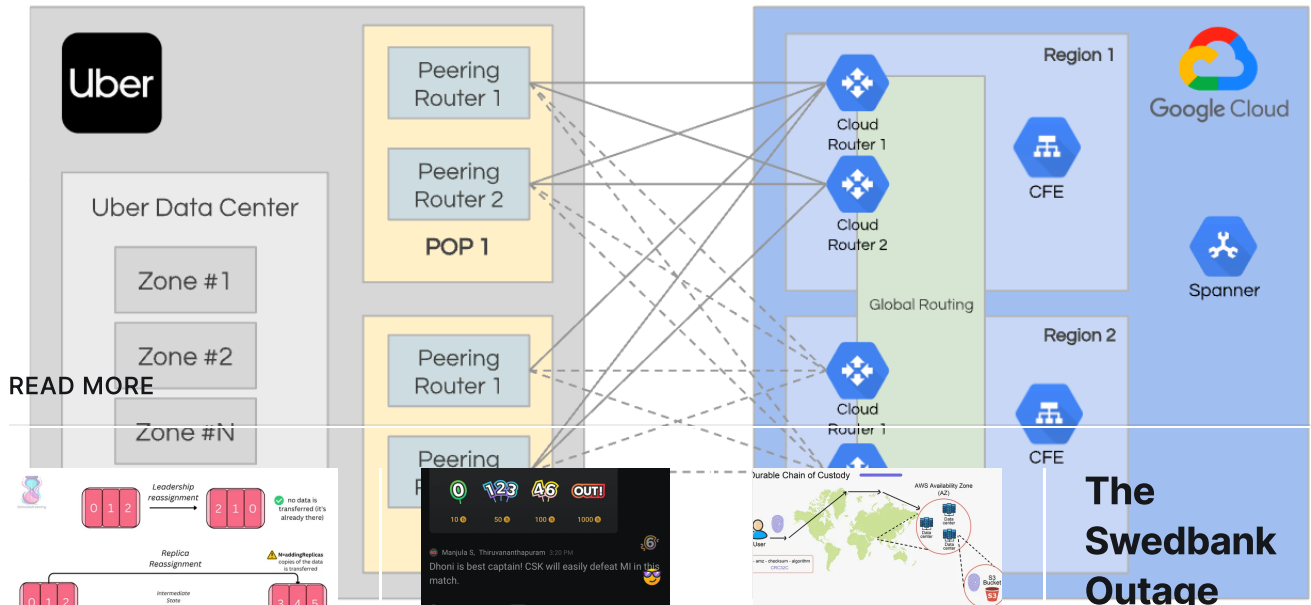


Some example services the Fulfillment Platform needed to support

So we made a bold bet and embarked on a journey to [rewrite the Fulfillment Platform from the ground up](#).

To satisfy the requirements of transactional consistency, horizontal scalability, and low operational overhead, we decided to leverage a NewSQL architecture. And opted to use [Google Cloud Spanner](#) as the primary storage engine.

As [lead Uber engineer Ankit Srivastava](#) puts it "as we scale and expand our global footprint, Spanner's scalability & low operational cost is invaluable. Prior to integrating Spanner, our data management framework demanded a lot of oversight and operational effort, escalating both complexity and expenditure."



Kafka 101

This is a guest article by Stanislav Kozlovski...

By HS Editor — 09 May 2024

Capturing A Billion Emojis

This blog post was written by...

By HS Editor — 26 Mar 2024

Behind AWS S3's Massive Scale

This is a guest article by...

By HS Editor — 06 Mar 2024

The Swedbank Outage shows that Change Controls don't work

This week I've been reading...

By Bruce Johnston

What else have we done?

Of course, our scaling story is never this simple. There's a countless number of things we've done over the years across all engineering and operations teams, including some of these larger initiatives.

Many of our most critical systems have their own rich history and evolution to address scale over the years. This includes our [API gateway](#), [fulfillment stack](#), [money stack](#), [real-time data intelligence platform](#), [geospatial data platform](#) (where we open-sourced [H3](#)), and building machine learning at scale through [Michelangelo](#).

We've introduced various layers of Redis caches. We've enabled powerful new frameworks to aid in scalable and reliable systems like [Cadence](#) (for writing fault-tolerant, long-running workflows).

High Scalability

We've built and leveraged data infrastructure that [enables long term growth](#), like how we [leveraged Presto](#) or [scaled Spark](#). Notably, we built [Apache Hudi](#) to power business critical data pipelines at low latency and high efficiency.

jamie@example.com

Subscribe

And finally, we continue to improve the [performance of our servers](#) with optimized hardware, advanced [memory](#) and [system tuning](#), and utilizing newer runtimes.

What's Next? Heading to the Cloud

Being a global company, Uber operated out of multiple on-prem data centers from the earliest days. But that introduced a number of challenges.

First, our server fleet had grown rapidly (over 250,000 servers) and the tooling and teams were always trying to keep up. Next, we have a large geographical footprint and need to regularly expand into more data centers and availability zones. Finally, with only on-prem machines, we were constantly needing to tune the size of our fleet.

We spent the last few years working towards making over [4000 of our stateless microservices portable](#). And to ensure our stack would work equally well across cloud and on-prem environments, we embarked on [Project Crane](#) to solve. This effort set Uber up well for the future. To learn more, watch lead Uber engineer [Kurtis Nusbaum's talk on how Crane](#) solves our scaling problems.

We now have plans to migrate a larger portion of our online and offline server fleet [to the Cloud](#) over the next few years!

Thanks to my many Uber friends and colleagues for reviewing this!

And if you enjoyed this, check out my blog post about [A Brief History of Scaling LinkedIn](#).

Member discussion

5 comments

Join the discussion

Become a member of **High Scalability** to start commenting.

Sign up now

Already a member? [Sign in](#)

Sort by: Best ▾



Chris Greeno · 11 Jun 2024

Be Great to update the broken image links :)

♡ 5 ↩ Reply

E

EenkheE · 15 Mar 2024

Brilliant!

♡ 2 ↩ Reply



Joseph Oun · 22 Oct 2025 (edited)

Great Post, Thanks for Sharing

maybe update broken image links? :)

♡ 0 ↩ Reply

L

leesa · 4 Sep 2025

"Really enjoyed the 'Brief History of Scaling Uber'—it's fascinating to see how Uber transformed from a Spanish-coded LAMP prototype into the world's largest mobility platform, serving over 130 million customers across thousands of backend services by evolving into scalable, service-oriented architecture using Node.js, Python, Redis, and more .

If you're looking to channel that same spirit of innovation into your own creative projects, this Flipaclip tutorial is a great place to start:

<https://flipaclip.upcomingweb.com/>

♡ 0 ↩ Reply

J **Jericho** · 7 Jan 2025

It's fascinating how scaling Uber required such precision and innovation, kind of like how every piece needs to fit perfectly in auto glass repairs. Speaking of which, if you're in need, MotorGlass has you covered! Visit

<http://www.motorglass.com.au/>

♡ 0 ↩ Reply